Lab 04 - GPG Key Management, Encryption and

Signing

Student: Nenad Damjanović | Environment: Kali Linux (VirtualBox)

Objective and environment

Use GNU Privacy Guard (GPG 2.4.9) to create asymmetric keys, export/import public keys, encrypt/decrypt messages, and create/verify digital signatures. The exercise also tests integrity after file modification and simulates secure correspondence with Alice.

Key generation and revocation

Generated an RSA 4096-bit signing/certification key with an RSA encryption subkey for nenad@test.com. The key was created on 19 October 2026 and expires on 20 October 2026; its expiration was intentionally left unchanged. A previously expired Ed25519 laboratory key was separately revoked.

Figure 1. GPG version and RSA key generation

```
┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ gpg --version
gpg (GnuPG) 2.4.9
libgcrypt 1.12.2
Copyright (C) 2025 g10 Code GmbH
License GNU GPL-3.0-or-later <https://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Home: /home/kali/.gnupg
Supported algorithms:
Pubkey: RSA, ELG, DSA, ECDH, ECDSA, EDDSA
Cipher: IDEA, 3DES, CAST5, BLOWFISH, AES, AES192, AES256, TWOFISH,
        CAMELLIA128, CAMELLIA192, CAMELLIA256
Hash: SHA1, RIPEMD160, SHA256, SHA384, SHA512, SHA224
Compression: Uncompressed, ZIP, ZLIB, BZIP2

┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ gpg --full-generate-key
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (sign only)
  (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 4096
Requested keysize is 4096 bits
Please specify how long the key should be valid.
         0 = key does not expire
      <n>  = key expires in n days
      <n>w = key expires in n weeks
      <n>m = key expires in n months
      <n>y = key expires in n years
Key is valid for? (0) 1ž
invalid value
Key is valid for? (0) 1
Key expires at Tue 20 Oct 2026 06:01:55 PM CEST
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: Neand Damjanović
Email address: nenad@test.com
Comment:
You are using the 'utf-8' character set.
You selected this USER-ID:
    "Neand Damjanović <nenad@test.com>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit?
Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: revocation certificate stored as '/home/kali/.gnupg/openpgp-revocs.d/1F5E9B5C5C53A8941CE23AC
8692B31616BFCB6C5.rev'
public and secret key created and signed.
```

Keyring inspection and public-key export

The gpg --list-keys and gpg --list-secret-keys outputs show the new RSA key and the revoked old key. The command gpg --armor --export nenad@test.com > public_key.asc created a distributable ASCII-armored public key.

Figure 2. Public and private key listing; revoked former key

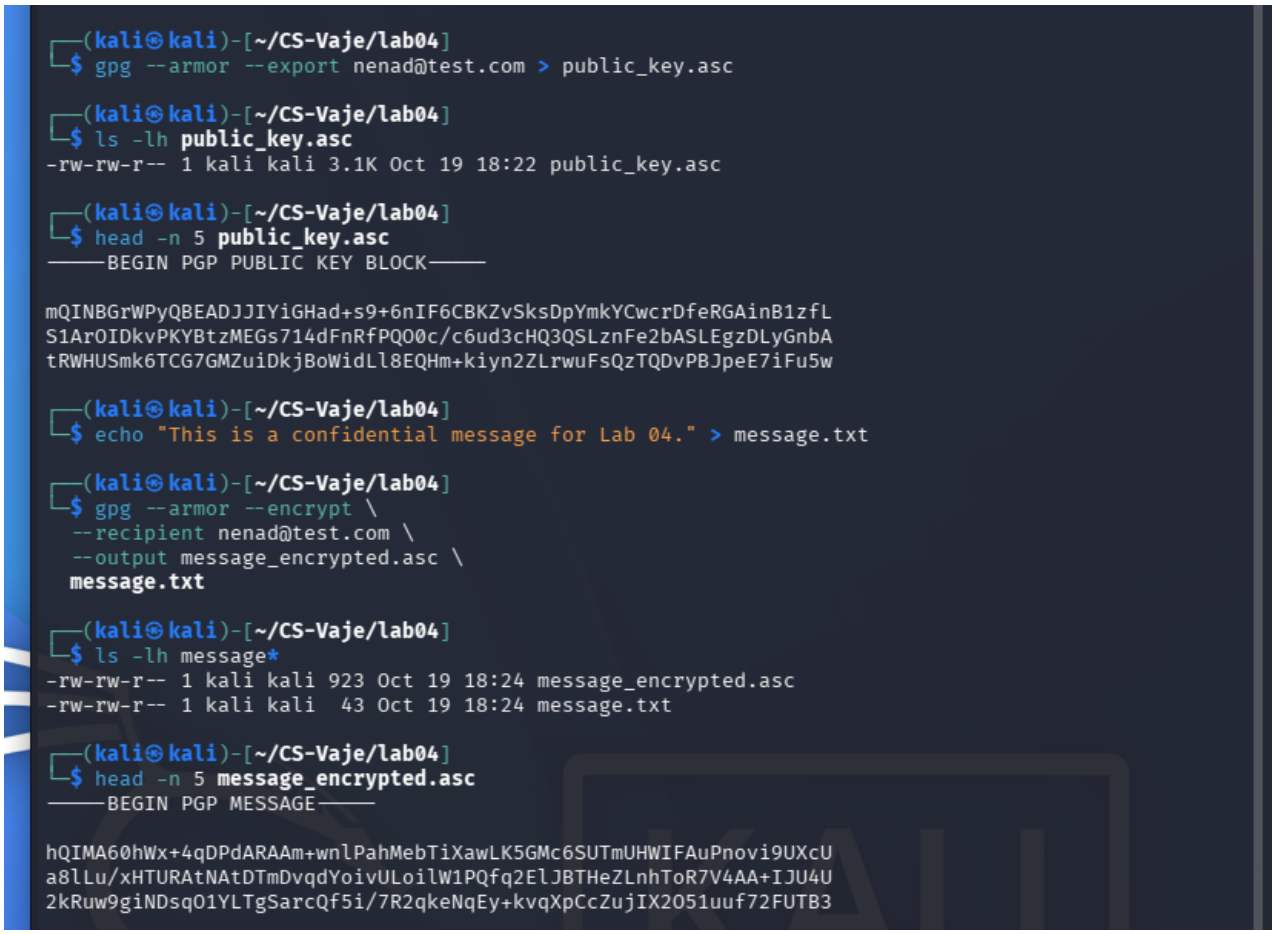


Figure 3. Public-key export and self-encryption

```
┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ gpg --list-keys
/home/kali/.gnupg/pubring.kbx
------------------------------
pub   ed25519 2026-10-14 [SC] [revoked: 2026-10-19]
      632CC7ADB2F5483E18F596421A5F761EC7D41D33
uid           [ revoked] Nenad Lab (SEEK AND DESTROY) <nenad.lab@example.com>

pub   rsa4096 2026-10-19 [SC] [expires: 2026-10-20]
      1F5E9B5C5C53A8941CE23AC8692B31616BFCB6C5
uid           [ultimate] Neand Damjanović <nenad@test.com>
sub   rsa4096 2026-10-19 [E] [expires: 2026-10-20]


┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ gpg --list-secret-keys
/home/kali/.gnupg/pubring.kbx
------------------------------
sec   ed25519 2026-10-14 [SC] [revoked: 2026-10-19]
      632CC7ADB2F5483E18F596421A5F761EC7D41D33
uid           [ revoked] Nenad Lab (SEEK AND DESTROY) <nenad.lab@example.com>

sec   rsa4096 2026-10-19 [SC] [expires: 2026-10-20]
      1F5E9B5C5C53A8941CE23AC8692B31616BFCB6C5
uid           [ultimate] Neand Damjanović <nenad@test.com>
ssb   rsa4096 2026-10-19 [E] [expires: 2026-10-20]
```

Encryption, decryption and signature verification

A test message was encrypted to the owner's public key and decrypted using the corresponding private key.

diff produced no output, proving that the recovered plaintext matched the original.

Figure 4. Successful decryption and file comparison

```
┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ gpg --armor --detach-sign \
  --local-user nenad@test.com \
  --output message.sig.asc \
  message.txt

┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ gpg --verify message.sig.asc message.txt
gpg: Signature made Mon 19 Oct 2026 06:26:41 PM CEST
gpg:                using RSA key 1F5E9B5C5C53A8941CE23AC8692B31616BFCB6C5
gpg:                issuer "nenad@test.com"
gpg: Good signature from "Neand Damjanović <nenad@test.com>" [ultimate]
```

The original text was signed using a detached GPG signature. Verification reported Good signature. After modifying a copy of the text, verifying the original signature against the changed copy reported BAD signature. This demonstrates detection of tampering, not prevention of modification.

Figure 5. Valid signature on the original message

```
┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ cp message.txt message_modified.txt

┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ echo "This message has been modified" >> message_modified.txt

┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ gpg --verify message.sig.asc message_modified.txt
gpg: Signature made Mon 19 Oct 2026 06:26:41 PM CEST
gpg:                using RSA key 1F5E9B5C5C53A8941CE23AC8692B31616BFCB6C5
gpg:                issuer "nenad@test.com"
gpg: BAD signature from "Neand Damjanović <nenad@test.com>" [ultimate]

┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ gpg --verify message.sig.asc message.txt
gpg: Signature made Mon 19 Oct 2026 06:26:41 PM CEST
gpg:                using RSA key 1F5E9B5C5C53A8941CE23AC8692B31616BFCB6C5
gpg:                issuer "nenad@test.com"
gpg: Good signature from "Neand Damjanović <nenad@test.com>" [ultimate]
```

Figure 6. Invalid signature for altered data; valid for the original

```
┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ gpg --output message_decrypted.txt --decrypt message_encrypted.asc
gpg: encrypted with rsa4096 key, ID AD215B1FB8A833DD, created 2026-10-19
      "Neand Damjanović <nenad@test.com>"

┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ cat message_decrypted.txt
This is a confidential message for Lab 04.

┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ diff message.txt message_decrypted.txt

┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$
```

Exchange of public keys and signed encryption

To simulate a separate recipient, an isolated alice_gpg keyring was created. Alice's public key was exported and imported into Nenad's keyring; the private key remained in Alice's isolated directory.

Figure 7. Alice's key creation and import of her public key



A message for Alice was encrypted using her public encryption key and simultaneously signed using Nenad's private signing key (gpg --armor --encrypt --sign).

Figure 8. Encryption and signing for Alice

```
┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ mkdir -p alice_gpg

┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ chmod 700 alice_gpg

┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ gpg --homedir ./alice_gpg --quick-generate-key \
  "Alice Lab <alice@example.com>" \
  default default 1y
gpg: keybox '/home/kali/CS-Vaje/lab04/./alice_gpg/pubring.kbx' created
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: /home/kali/CS-Vaje/lab04/./alice_gpg/trustdb.gpg: trustdb created
gpg: directory '/home/kali/CS-Vaje/lab04/./alice_gpg/openpgp-revocs.d' created
gpg: revocation certificate stored as '/home/kali/CS-Vaje/lab04/./alice_gpg/openpgp-revocs.d/2029
E94FBFE406FA6B9A3F4994A94B058795CAD7.rev'
public and secret key created and signed.

pub   ed25519 2026-10-19 [SC] [expires: 2027-10-19]
      2029E94FBFE406FA6B9A3F4994A94B058795CAD7
uid                      Alice Lab <alice@example.com>
sub   cv25519 2026-10-19 [E]


┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ gpg --homedir ./alice_gpg \
  --armor --export alice@example.com > alice_public.asc

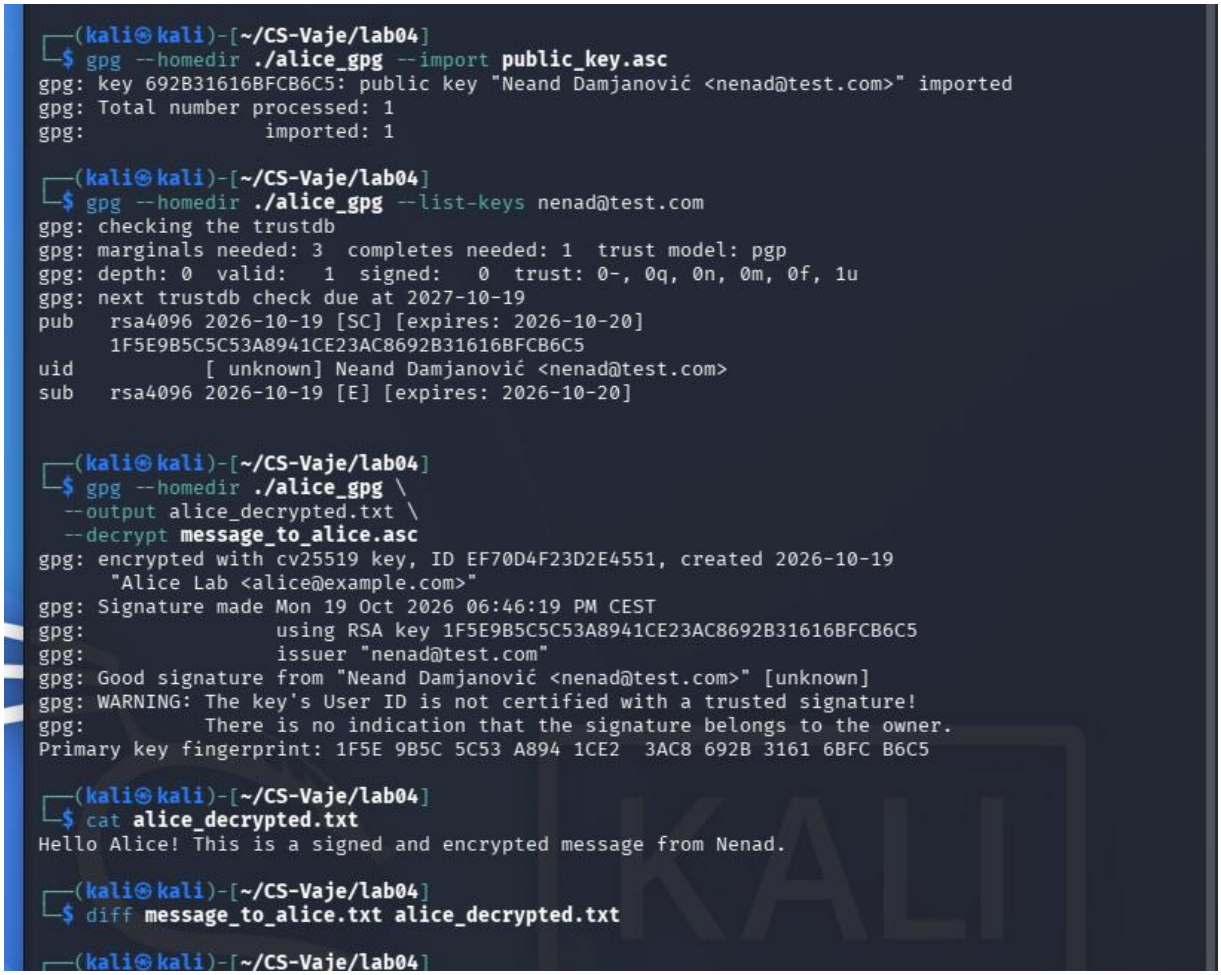
┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ gpg --import alice_public.asc
gpg: key 94A94B058795CAD7: public key "Alice Lab <alice@example.com>" imported
gpg: Total number processed: 1
gpg:               imported: 1

┌──(kali㉿kali)-[~/CS-Vaje/lab04]
└─$ gpg --list-keys alice@example.com
pub   ed25519 2026-10-19 [SC] [expires: 2027-10-19]
      2029E94FBFE406FA6B9A3F4994A94B058795CAD7
uid           [ unknown] Alice Lab <alice@example.com>
sub   cv25519 2026-10-19 [E]
```

Recipient decryption and trust verification

Alice imported Nenad's public key, decrypted the signed encrypted message in her own GPG environment, and received Good signature [unknown]. The plaintext matched the original by diff. The [unknown] trust label means the key-to-person identity was not independently certified; signature validity alone does not prove a real-world identity.

Figure 9. Alice decrypts and verifies the signed message



Questions and answers

Encryption vs. signing: Encryption provides confidentiality for the intended recipient. Signing supports verification of message integrity and the signing key, but does not hide content.

Role of keys: The recipient's public key encrypts and their private key decrypts; the sender's private key signs and the sender's public key verifies. Public-key identity must be independently checked.

What if a file is modified? The demonstrated signed plaintext modification produced BAD signature. A separate direct corruption test of encrypted ciphertext was not performed.

Security and conclusion: All operations used local test identities. Never publish private keys, passphrases, the alice_gpg/ directory, or revocation certificates. GPG successfully demonstrated confidentiality, integrity checking, and signed encrypted communication.

**ChatGPT is used to create a report from my own work and screenshots*